Grade Management System (Python + Docker)

Project Goal

You will **plan and develop** a **grade management web application**.

- **Students** can log in and view their grades.
- **Professors** can log in and create/update grades for students.

The application must be implemented using **Python**, a **relational database**, and **Docker** for containerization.

Important: Before implementation, you must perform **structured planning**. This includes:

- Creating **Personas**
- Writing **User Stories**
- Prioritizing features using **MoSCoW**
- Creating a **Project Plan** in **MS Planner**
- Conducting **Research** on unknown technologies, frameworks, and workflows

Your **research process must be documented** in a written document (PDF) and submitted as part of the deliverables.

Non-functional Requirements

- Secure password hashing
- Code must be clean and well-structured
- The entire system must start using one command: docker compose up
- A README must explain how to run the system

User Stories

Write **at least 12 User Stories**. Each User Story must include **acceptance criteria**.

Example:

As a Student, I want to log in so that I can see my grades. **Acceptance Criteria:** Valid credentials grant access; invalid credentials show an error; logout exists.

After writing all User Stories, perform **MoSCoW prioritization**.

Our **User Stories** and **MoSCoW prioritization** must be written in a **Document (PDF): Functional Requirements.**

This document will be graded.

Research Requirements

Before coding, you must research:

- Suitable Python frameworks
- How to connect Python to a SQL database
- How to securely hash passwords
- How to use Docker and Docker Compose

Your research must be written in a **Research Document (PDF)** containing:

- Sources used (links, tutorials, documentation)
- Key findings and decisions
- Why you chose certain technologies

This document will be graded.

MS Planner Setups

Create a board with at least the following columns:

- Backlog
- Ready
- In Progress
- Review
- Done

Each User Story must become a **Task** with:

- Clear description
- Acceptance criteria
- Assigned team member
- Due date
- Progress updates

MS Planner Setups

Planning Document (PDF)

- Personas
- User Stories + MoSCoW Prioritization
- System Architecture Sketch
- ER Diagram

Research Document (PDF) (required)

- Description of investigated frameworks/tools
- Justification of architectural decisions

MS Planner Board Export or screenshots

Source Code Repository (ZIP)

- docker-compose.yml
- Python application
- README.md

Demonstration – live demo (3–5 minutes)

Reflection (1–2 pages)

- Team collaboration
- What worked well / challenges

Grading (100 Points)

- Planning Documentation: 30
- Research Documentation: 20
- Implementation & Code Quality: 30
- Planner Board Usage: 10
- Demo & Reflection: 10

Submission: 17.11.2025 – 16:30

Presentation: 24.11.2025 – 14:45